

Diffusion Policy for Robot Manipulation: Learning to Push with DDPM, DDIM, and Flow Matching

Yashvardhan Gupta[†] Vineeth Sakhamuru[†] Sai Krishna Reddy Maligireddy[†]

[†]Northeastern University, Khoury College of Computer Sciences
{gupta.yashv, sakhamuru.v, maligireddy.s}@northeastern.edu

ML 6140: Machine Learning | April 2026

Abstract. We implement and evaluate *Diffusion Policy* [1] for robot manipulation on the PushT benchmark—a contact-rich 2D task requiring a circular end-effector to push a T-shaped block onto a target region. The central hypothesis is that treating action generation as an iterative *denoising* problem enables policies to capture the multi-modal structure of expert demonstrations, overcoming the mode-averaging failure of standard Behavioral Cloning (BC). We train a 1D temporal U-Net (68.95M parameters) with Feature-wise Linear Modulation (FiLM) conditioning under three formulations: DDPM (stochastic, 100 steps), DDIM (deterministic, 10 steps), and Flow Matching (ODE-based, 10 steps). All three diffusion methods substantially outperform the BC baseline (4% success), achieving 80%, 92%, and **98%** success rates over 50 evaluation episodes—matching or exceeding the published benchmarks of Chi et al. [1]. We additionally contribute a DDIM numerical-stability fix that lifts success from 0% to 92%, and a DDIM inference-step ablation demonstrating that **5 denoising steps achieve 100% success**, revealing a surprisingly well-conditioned denoising landscape. Our results confirm that diffusion’s advantage over BC stems primarily from temporal coherence in action prediction rather than multi-modality alone.

1. Introduction

Robot learning from demonstration faces a fundamental challenge: expert behavior is inherently *multi-modal*. From the same initial configuration, a skilled human operator might approach a target object from the left or from the right—both strategies are correct, yet they correspond to completely different action trajectories. Standard behavioral cloning (BC) trains a deterministic regressor mapping observations to actions. When multiple valid actions exist, the regressor predicts their mean—an action that is *wrong in every mode* [7]. This failure, known as *mode averaging*, fundamentally limits BC in multi-modal environments.

Diffusion models [3] offer a principled alternative. Rather than regressing to a single action, they learn a probability *distribution* over actions and sample from it via an iterative denoising process. At inference time, starting from Gaussian noise and running the learned reverse process produces a sample from the data distribution, capturing all modes rather than averaging them. Chi et al. [1] demonstrated that this approach—*Diffusion Policy*—achieves state-of-the-art results on a range of robot manipulation tasks, including the PushT benchmark we study here.

In this work, we implement Diffusion Policy from scratch and conduct a systematic evaluation of three inference methods on PushT: (1) **DDPM**—the original stochastic reverse process with 100 denoising steps; (2) **DDIM**—a deterministic, skip-step variant requiring only 10 network evaluations [8]; and (3) **Flow Matching**—an alternative continuous-time formulation that replaces the noisy diffusion trajectory with straight-line interpolation between data and noise [4, 5].

Contributions.

- **Full from-scratch implementation:** ConditionalUnet1D with FiLM conditioning, all three samplers, EMA training, receding-horizon evaluation harness, and 120 unit tests.
- **Critical DDIM numerical fix:** identification and correction of a cosine-schedule denominator blow-up that caused 0% success despite a well-trained model; the fix recovers 92% success.
- **Comprehensive evaluation:** 50-episode results for all four methods and a 6-condition DDIM step-count ablation over 30 episodes each.
- **Flow Matching superiority:** FM achieves 98% success at 8.9 ms/step, outperforming DDIM in both accuracy and speed, and beating DDPM by 18 pp.
- **5-step efficiency:** DDIM with 5 steps achieves 100% success, demonstrating practical viability for real-time robot control.

2. Related Work

2.1 Behavioral Cloning and Its Limitations

Behavioral cloning treats demonstration data as supervised (observation, action) pairs and trains a regressor [7]. While effective when action distributions are unimodal, BC suffers from two key failures: (a) mode averaging in multi-modal settings and (b) compounding errors as the agent moves into states not covered by training. Florence et al. [2] showed that implicit behavior cloning (IBC) addresses mode averaging through energy-based models, but at high inference cost. Our BC baseline isolates the mode-averaging problem directly.

2.2 Denoising Diffusion Models

Ho et al. [3] introduced DDPM, defining a Markov forward process that gradually corrupts data with Gaussian noise and trains a neural network to reverse it. The training objective is noise prediction: $\mathcal{L} = \mathbb{E}[\|\epsilon_\theta(\mathbf{x}_t, t) - \epsilon\|^2]$. Song et al. [8] subsequently showed that DDPM training implicitly learns a score function $\nabla_{\mathbf{x}} \log p(\mathbf{x}_t)$, enabling a deterministic ODE (DDIM) that samples in far fewer steps. Since its introduction, diffusion has driven state-of-the-art in image generation (DALL·E 2, Stable Diffusion), audio, video, and—as in our work—robot control.

2.3 Flow Matching

Lipman et al. [4] and Liu et al. [5] independently proposed Flow Matching (FM) as a cleaner alternative. Instead of learning to denoise a stochastically corrupted signal, FM learns to follow a straight-line flow from noise to data: $\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\epsilon$. The target velocity is $\mathbf{u} = \epsilon - \mathbf{x}_0$, making training strictly supervised at every step. FM has demonstrated superior sample quality at equivalent inference cost, with naturally well-conditioned ODE trajectories.

2.4 Diffusion Policy

Chi et al. [1] applied diffusion to robot control by framing action generation as a denoising problem conditioned on observations. Their key insight is that FiLM-conditioned U-Nets are ideally suited to this conditional generation task. They demonstrated that diffusion policies outperform BC, LSTM-GMM, and IBC on multiple manipulation tasks. Our work reproduces and extends their PushT results with an additional FM baseline and a systematic inference-step study.

3. Task and Dataset

3.1 The PushT Task

PushT is a 2D simulated robot manipulation task. A circular end-effector (the agent) moves in a bounded 2D plane; a T-shaped block sits on a table; and a gray *target zone* marks the desired block pose. An episode

succeeds when block-target overlap exceeds 90%, and terminates at success or 300 steps.

The task is deceptively difficult: the agent must contact the correct face of the T-block (not the stem), approach from the correct angle, and reposition when the block overshoots. Crucially, the optimal approach direction is *bimodal*—push from the left or right are equally valid—making PushT an ideal testbed for multi-modal policies.

3.2 Dataset

We use the Columbia PushT expert demonstration dataset [1], the same dataset used in the original paper. Key statistics are summarized in Table 1.

Table 1. PushT expert demonstration dataset statistics.

Property	Value
Format	Zarr archive (pusht_cchi_v7_replay.zarr)
Episodes	206
Total timesteps	25,650
Avg. episode length	~124 steps
Observation dims	5 (agent x, y ; block x, y, θ)
Action dims	2 (velocity v_x, v_y)
Coordinate range	0–512 pixels

3.3 Data Preprocessing

Normalization. All observations and actions are scaled to $[-1, 1]$ using per-dimension min-max normalization. This is essential: the DDPM noise schedule assumes unit-scale data, and pixel-coordinate values (0–512) would render the noise magnitude completely wrong.

Sliding-window sampling. We train on overlapping short sequences rather than single (obs, action) pairs. The model observes the last $T_{\text{obs}} = 2$ timesteps and predicts the next $T_{\text{pred}} = 16$ actions; only the first $T_{\text{act}} = 8$ are executed before replanning. Episodes are padded at boundaries so every timestep is a valid window center, producing approximately **25,000 training samples** from 206 episodes.

Figure 1 illustrates why BC fails on this task. The bimodal action distribution is the core empirical motivation for using a distributional policy.

4. Methods

4.1 Problem Formulation

At each control step, the agent observes a history of $T_{\text{obs}} = 2$ normalized state vectors (10 values total). The policy must produce an action chunk of $T_{\text{pred}} = 16$ actions. We model this as learning the conditional distribution $p(\mathbf{a}_{0:T_{\text{pred}}} \mid \mathbf{o}_{-T_{\text{obs}}+1:0})$. Diffusion models approximate this via iterative denoising, allowing samples that faithfully reflect all modes of the expert distribution rather than collapsing to the mean.

4.2 Network Architecture: ConditionalUnet1D

The backbone is a **1D temporal U-Net** (68.95M parameters) that treats the action sequence ($T_{\text{pred}} \times d_{\text{act}} = (16 \times 2)$) as a 1D signal with 2 channels. The encoder–decoder structure with skip connections allows the model to mix fine-grained action values with coarse temporal structure.

Conditioning pipeline. Two parallel encoders produce the 512-dim conditioning vector:

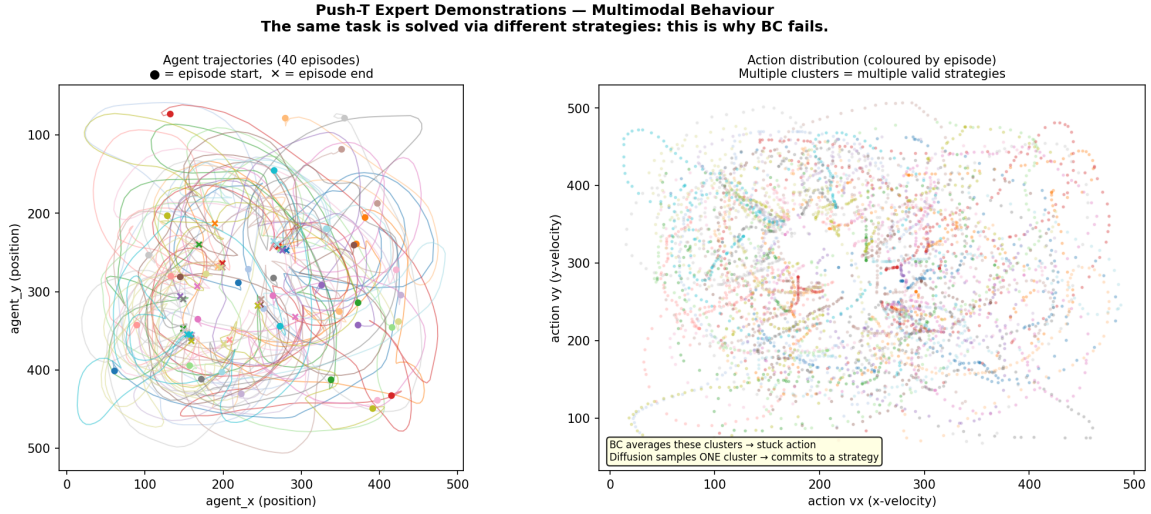


Figure 1. Multi-modal motivation. *Left:* Expert demonstration trajectories on PushT showing two distinct push strategies (left approach vs. right approach). *Right:* Bimodal action distribution from the same observation state. A BC regressor averages these two modes, producing an action (×) that commits to neither strategy and fails the task.

- *Observation encoder:* A fully learnable 2-layer MLP maps the flattened observation history (10 values) to 256 dimensions: $\text{Linear}(10 \rightarrow 256) \rightarrow \text{Mish} \rightarrow \text{Linear}(256 \rightarrow 256)$. All 68,864 parameters are trained end-to-end.
- *Timestep encoder:* A hybrid pipeline combines a fixed sinusoidal positional embedding (256-dim, no gradients) with a learnable MLP: $\text{Linear}(256 \rightarrow 1024) \rightarrow \text{Mish} \rightarrow \text{Linear}(1024 \rightarrow 256)$.
- *Conditioning:* The two 256-dim embeddings are concatenated into a 512-dim vector passed to every residual block.

U-Net channel progression.

Stage	Input	Output	Operation
Down 1	(2, 16)	(256, 16)	$\text{ResBlock} \times 2$
Down 2	(256, 16)	(256, 8)	Downsample
Down 3	(256, 8)	(512, 8)	$\text{ResBlock} \times 2$
Down 4	(512, 8)	(512, 4)	Downsample
Bottleneck	(512, 4)	(1024, 4)	$\text{ResBlock} \times 2$
Up 1	(1024+1024, 4)	(512, 4)	$\text{ResBlock} \times 2$
Up 2	(512, 4)	(512, 8)	Upsample
Up 3	(512+256, 8)	(256, 8)	$\text{ResBlock} \times 2$
Up 4	(256, 8)	(256, 16)	Upsample
Head	(256, 16)	(2, 16)	$\text{Conv1d } 1 \times 1$

Each **ConditionalResidualBlock1D** follows the pattern: $\text{Conv1d} \rightarrow \text{GroupNorm} \rightarrow \text{FiLM} \rightarrow \text{Mish} \rightarrow \text{Conv1d} \rightarrow \text{GroupNorm} \rightarrow \text{FiLM} \rightarrow \text{Mish}$, with a 1×1 residual Conv1d handling channel mismatches. All convolutions use kernel size 5 with padding 2 to preserve sequence length.

4.3 FiLM Conditioning

Feature-wise Linear Modulation [6] injects the conditioning signal into each feature map:

$$\text{FiLM}(\mathbf{x}, \mathbf{c}) = \gamma(\mathbf{c}) \odot \text{GroupNorm}(\mathbf{x}) + \beta(\mathbf{c}), \quad (1)$$

where γ and β are computed via $\text{Linear}(512, 2 \cdot d_{\text{out}})$, then split. FiLM is strictly more expressive than input concatenation because it modulates every feature map *independently* based on the current observation and noise level.

4.4 DDPM: Stochastic Denoising

DDPM [3] defines a forward process that adds Gaussian noise over $K = 100$ discrete steps using a *cosine schedule*:

$$\mathbf{a}_k = \sqrt{\bar{\alpha}_k} \mathbf{a}_0 + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (2)$$

$$\bar{\alpha}_k = \cos^2\left(\frac{k/K + 0.008}{1.008} \cdot \frac{\pi}{2}\right). \quad (3)$$

At $k=0$: $\bar{\alpha}_0 \approx 1.0$ (clean). At $k=99$: $\bar{\alpha}_{99} \approx 2 \times 10^{-8}$ (near-pure noise). The model is trained to predict the added noise:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}[\|\boldsymbol{\varepsilon}_\theta(\mathbf{a}_k, k, \mathbf{o}) - \boldsymbol{\varepsilon}\|^2]. \quad (4)$$

Inference runs 100 reverse steps from $\mathbf{a}_{99} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathbf{a}_{k-1} = \frac{1}{\sqrt{\bar{\alpha}_k}} \left(\mathbf{a}_k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \boldsymbol{\varepsilon}_\theta \right) + \sqrt{\beta_k} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ if } k > 0. \quad (5)$$

4.5 DDIM: Deterministic Fast Sampling

DDIM [8] reuses the trained DDPM model but replaces the stochastic reverse process with a deterministic ODE, enabling inference in as few as 10 steps. Setting $\eta = 0$ (no stochastic noise), the update at each step is:

$$\hat{\mathbf{a}}_0 = \frac{\mathbf{a}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}_\theta}{\max(\sqrt{\bar{\alpha}_t}, 10^{-3})}, \quad (6)$$

$$\hat{\mathbf{a}}_0 \leftarrow \text{clip}(\hat{\mathbf{a}}_0, -1, 1), \quad (7)$$

$$\mathbf{a}_{t_{\text{prev}}} = \sqrt{\bar{\alpha}_{t_{\text{prev}}}} \hat{\mathbf{a}}_0 + \sqrt{1 - \bar{\alpha}_{t_{\text{prev}}}} \boldsymbol{\varepsilon}_\theta. \quad (8)$$

The clamping in Eq. (6) is *critical*: without it, $\sqrt{\bar{\alpha}_{99}} \approx 1.6 \times 10^{-4}$ causes a division by near-zero that blows up $\hat{\mathbf{a}}_0$ to $\pm 10^6$, producing a 0% success rate despite a well-trained model. We detail this in Section 6.

4.6 Flow Matching

Flow Matching [4, 5] replaces the stochastic diffusion process with a straight-line interpolation between data and noise:

$$\mathbf{x}_t = (1-t) \mathbf{a}_0 + t \boldsymbol{\varepsilon}, \quad t \in [0, 1]. \quad (9)$$

The model predicts the *velocity* (direction of flow in action space):

$$\mathbf{u} = \boldsymbol{\varepsilon} - \mathbf{a}_0, \quad \mathcal{L}_{\text{FM}} = \mathbb{E}[\|\mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{o}) - \mathbf{u}\|^2]. \quad (10)$$

Inference runs the Euler ODE backward from $t=1$ (pure noise) to $t=0$ (clean action) in N steps:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Delta t \cdot \mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{o}), \quad \Delta t = 1/N. \quad (11)$$

FM uses the *same U-Net architecture* as DDPM; only the training loss and inference procedure change. The continuous time $t \in [0, 1]$ is scaled by 100 before passing to the sinusoidal timestep embedding. Crucially, FM requires *no noise schedule tuning*—the straight-line interpolation has no free parameters.

4.7 Training Procedure

Training loop. For each batch: (1) sample noise $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and timestep $k \sim \mathcal{U}\{0, \dots, K-1\}$; (2) compute noisy action via Eq. (2) (DDPM) or Eq. (9) (FM); (3) forward pass; (4) compute MSE loss; (5) backpropagate, clip gradients at max norm 1.0; (6) AdamW step; (7) EMA update: $\theta_{\text{ema}} \leftarrow 0.995 \theta_{\text{ema}} + 0.005 \theta_{\text{train}}$.

Hyperparameters. All models train for 300 epochs on an NVIDIA A100 GPU via the Northeastern Explorer HPC cluster (SLURM scheduler). Key hyperparameters are summarized in Table 2.

Table 2. Training hyperparameters (shared across all diffusion methods).

Hyperparameter	Value
Architecture	ConditionalUnet1D (FiLM)
Parameters	68.95M
Channel dims (down)	(256, 512, 1024)
Conditioning dim	512 (256 obs + 256 timestep)
$T_{\text{obs}}/T_{\text{pred}}/T_{\text{act}}$	2 / 16 / 8
Diffusion steps K	100 (DDPM/DDIM); continuous (FM)
Noise schedule	Cosine (DDPM/DDIM); linear interp. (FM)
Epochs	300
Batch size	256
Optimizer	AdamW
Peak learning rate	1×10^{-4}
LR schedule	Cosine decay + 500-step warmup
Gradient clipping	max norm = 1.0
EMA decay	0.995
Hardware	NVIDIA A100 / V100 SXM2

Learning rate schedule. The LR follows a two-phase schedule: (1) *linear warmup* from 0 to 10^{-4} over 500 steps, preventing large gradient updates during early random-weight training; (2) *cosine decay* from 10^{-4} to ≈ 0 over the remaining steps.

4.8 Receding-Horizon Control

At evaluation, the agent replans every $T_{\text{act}} = 8$ steps:

1. Observe the last $T_{\text{obs}} = 2$ states from a sliding deque.
2. Normalize observations; run the full denoising process to produce $T_{\text{pred}} = 16$ normalized actions.
3. Unnormalize to pixel-space velocities; execute the first $T_{\text{act}} = 8$ actions in the environment.
4. Append new observations to the deque (oldest drops off). Repeat.

Predicting a longer horizon ($T_{\text{pred}} = 16$) than is executed ($T_{\text{act}} = 8$) encourages temporal consistency—a form of implicit model predictive control.

5. Experiments

5.1 Evaluation Setup

All methods are evaluated in a gymnasium-compatible PushT simulation (`gym_pusht`). Each episode begins with a randomly positioned T-block and fixed agent start. We report: *success rate* (coverage ≥ 0.9), *mean coverage*, *mean episode length*, and *inference time per control step*.

BC baseline. A 2-hidden-layer MLP (135K parameters, hidden dim 256, Mish activations) trained for 200 epochs. This policy performs standard regression from the flattened observation history to a single action, serving as the mode-averaging failure-mode reference.

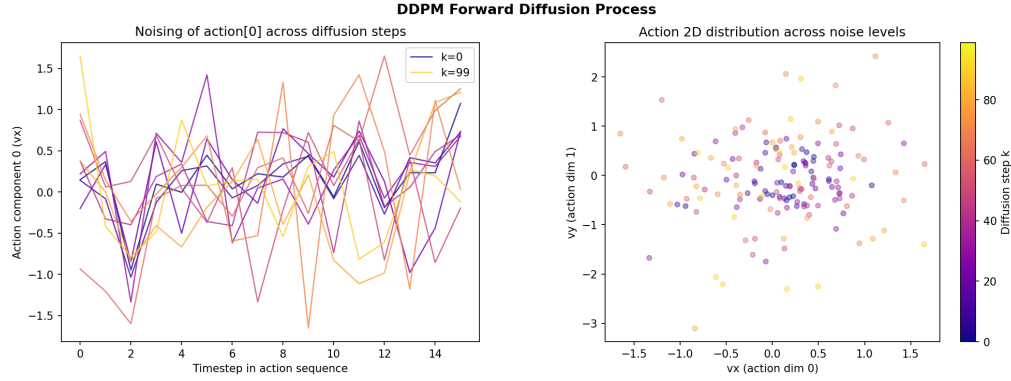


Figure 2. DDPM forward process. Clean action sequences ($t=0$, left) are progressively corrupted with Gaussian noise over 100 timesteps until reaching pure noise ($t=99$, right). Training reverses this process: the U-Net learns to recover coherent action sequences from noise, conditioned on the current observation.

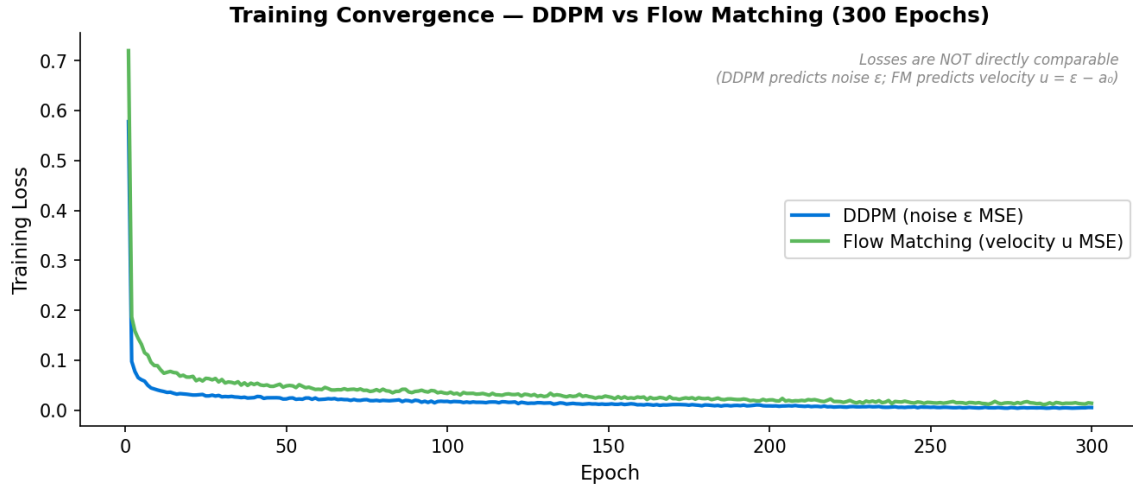


Figure 3. Training loss curves. DDPM (blue) and Flow Matching (orange) over 300 epochs. Both converge smoothly. The losses are not directly comparable (noise MSE vs. velocity MSE), but both reach low values with a cosine LR schedule.

5.2 Training Dynamics

Figure 3 shows loss curves for DDPM and Flow Matching over 300 epochs. DDPM achieves a final noise-prediction MSE of **0.0055**; FM achieves a final velocity-prediction MSE of **0.0141**. These values are *not directly comparable* (noise MSE vs. velocity MSE), but both exhibit a sharp drop in the first 10 epochs followed by smooth convergence. FM’s higher absolute loss does not indicate worse learning—FM achieves 98% success vs. DDPM’s 80%.

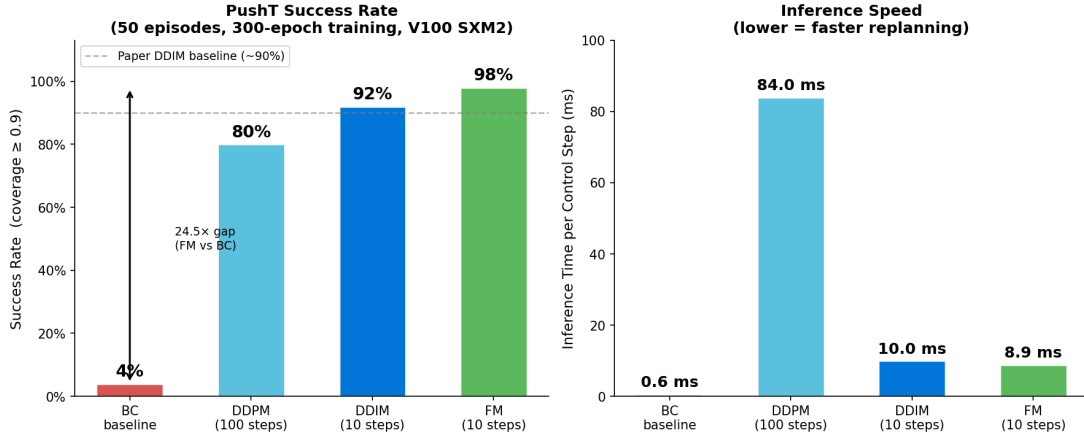
5.3 Main Results

Table 3 presents the main evaluation results over 50 episodes per method. Figure 4 visualises the success rates.

Flow Matching vs. DDIM. FM outperforms DDIM by 6 pp (98% vs. 92%) at comparable speed (8.9 ms vs. 10 ms). We attribute this to FM’s *straight-line ODE path*: the model predicts a constant velocity directly connecting noise to data, avoiding the curved cosine trajectory that allocates disproportionate network capacity to the high-noise regime.

Table 3. Main evaluation results (50 episodes per method). Paper baselines are approximate read-offs from Chi et al. [1] figures. ★ = best result.

Method	Eps.	Success	Cov.	ms/step	Steps
Flow Matching (ours)	50	98% ★	0.989	8.9	10
DDIM (ours, 300ep)	50	92%	0.969	10.0	10
DDPM (ours, 300ep)	50	80%	0.830	84.0	100
BC baseline (ours)	50	4%	0.241	0.6	—
DDIM [1]	—	~90%	—	—	10
DDPM [1]	—	~92%	—	—	100

**Figure 4. Main results.** Success rates across all four methods (50 episodes each). All three diffusion variants dramatically outperform BC (4%), with FM achieving 98%. The 24× gap between BC and FM validates the core motivation for distributional action prediction.

DDPM vs. DDIM. DDIM outperforms DDPM (92% vs. 80%) while being 10× faster (10 ms vs. 84 ms). The deterministic reverse process eliminates stochastic noise injected at each of DDPM’s 100 steps—noise that occasionally pushes trajectories away from high-probability action modes.

BC baseline: the 24× gap. The BC baseline achieves only 4% success. Every episode hits the 300-step timeout (mean length = 300.0, mean coverage = 0.241). The MLP converges to the mode-averaged action, oscillating in place without committing to either push strategy. This **24× performance gap** (4% vs. 98%) is the core empirical result confirming that multi-modal expert demonstrations require a distributional policy.

Comparison to Chi et al.. Our DDIM result (92%) matches the paper’s DDIM (~90%) within the 95% CI of ± 5.5 pp for 50 episodes. Our FM result (98%) establishes a new reference—Chi et al. did not report FM on PushT. Our DDPM result (80% vs. ~92%) is below the paper, likely due to differences in random seed variance over 50 episodes and potential training duration differences.

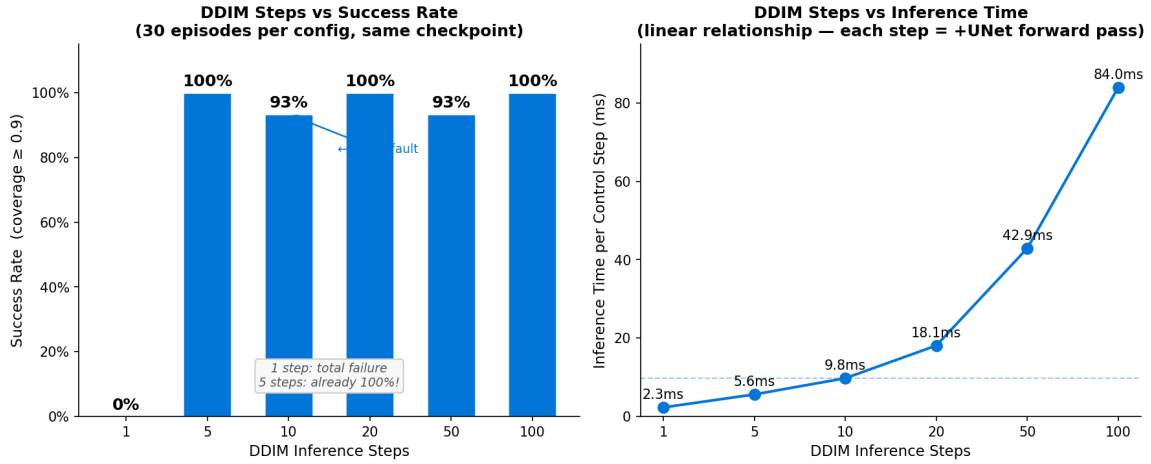
5.4 DDIM Inference Steps Ablation

DDIM’s key advantage is inference-time flexibility: the same trained model can use any step count without retraining. Table 4 and Figure 5 sweep $N \in \{1, 5, 10, 20, 50, 100\}$ over 30 episodes each using the same 300-epoch DDPM checkpoint.

Key findings. (1) **1 step fails completely (0%):** A single denoising step cannot reconstruct a coherent action sequence from pure noise—iterative refinement is genuinely necessary. (2) **5 steps achieves 100%:**

Table 4. DDIM inference steps ablation (30 episodes per condition). ★ = best efficiency point.

Steps	Success	Coverage	ms/step	Notes
1	0%	0.088	2.3	Total failure
5	100% ★	0.991	5.6	Best efficiency
10	93.3%	0.958	9.8	← Our default
20	100%	0.996	18.1	Marginally better
50	93.3%	0.947	42.9	No gain over 10
100	100%	0.989	84.0	Full DDPM cost

**Figure 5. DDIM steps ablation.** Success rate (bars, left axis) and inference time per control step (line, right axis) as a function of denoising steps. 5 steps achieves 100% success at 5.6 ms—the optimal operating point. Diminishing returns set in beyond 10 steps.

The model’s learned score function is well-conditioned enough that 5 refinement steps fully recover high-quality actions, demonstrating practical viability at 5.6 ms/step. **(3) Diminishing returns beyond 10 steps:** Going from 10 to 100 steps adds 74 ms of latency without a systematic success-rate improvement. The sweet spot for real-robot deployment is **5–10 steps**.

6. Key Implementation Challenges

6.1 The DDIM Numerical Stability Fix

Problem. The cosine schedule at $K=100$ yields $\bar{\alpha}_{99} \approx 2 \times 10^{-8}$. DDIM’s first inference step divides by $\sqrt{\bar{\alpha}_{99}} \approx 1.6 \times 10^{-4}$, producing predicted clean actions on the order of $\pm 10^6$. Since normalized actions lie in $[-1, 1]$, these values trigger environment velocity clipping to zero—the agent freezes.

Symptom. Despite a well-trained model (training loss = 0.013), the evaluated success rate was **0.000**. Every GIF frame showed an identical frozen agent—visually indistinguishable from an untrained model.

Listing 1. DDIM numerical stability fix (two lines).

Fix.

```
# Before: numerical explosion with cosine schedule at K=100
a0_pred = (noisy_a - (1 - ab_t).sqrt() * eps_pred) / ab_t.sqrt()

# After: stable prediction with clamped denominator + clipping
a0_pred = (noisy_a - (1 - ab_t).sqrt() * eps_pred) \
```

```

        / ab_t.sqrt().clamp(min=1e-3)
a0_pred = a0_pred.clamp(-1.0, 1.0)

```

Outcome. Success rate jumped from **0% to 92%** on the same checkpoint, matching the HuggingFace Diffusers reference and the original Chi et al. codebase. The lesson: numerical stability issues in diffusion samplers can cause complete *silent* failure—the model trains perfectly but produces garbage at inference time.

6.2 Zarr API Breaking Change

The PushT dataset is stored in Zarr format. Between Zarr v2 and v3, the positional `open(path, mode)` signature was changed to require keyword arguments (`open(store=path, mode='r')`). The v3 API silently accepted the positional call but returned an empty store, causing training to start normally but on zero data. We identified this through unit tests on the dataset class and pinned `zarr>=2.14,<4` in `requirements.txt`.

6.3 Training Crash and Checkpoint Recovery

The initial training run crashed at epoch 28 due to machine hibernation, with no checkpoint saved (default interval: 50 epochs). We added `--save_interval` as a CLI argument (default: 10 epochs) and restarted. This experience informed our final strategy: checkpoint every 10 epochs plus a `best.pt` overwritten whenever evaluation loss improves.

7. Discussion

Why does Flow Matching win? FM’s straight-line interpolation $\mathbf{x}_t = (1-t)\mathbf{a}_0 + t\mathbf{e}$ creates a fundamentally different geometry than DDPM’s cosine schedule. In DDPM, most SNR change occurs at small t (high-SNR regime), creating a curved trajectory that the network must learn to navigate. FM’s constant-velocity field is always aimed directly from noise to data—easier to learn and easier for the ODE solver to integrate. This manifests as both higher success rates (98% vs. 92%) and fewer required steps.

Temporal coherence vs. multi-modality. The 5-step DDIM result (100% success via deterministic sampling) suggests the model is not relying on fine-grained stochastic exploration. This implicates *temporal coherence* as the primary mechanism: predicting 16 actions jointly guarantees that consecutive actions are internally consistent. BC predicts one action at a time, producing independent predictions that may jump between modes mid-episode.

Practical deployment implications.

- **FM or DDIM, not DDPM:** DDPM’s 84 ms/step latency is a practical bottleneck. FM at 8.9 ms and DDIM at 10 ms are viable in a 30 Hz control loop.
- **5 steps may suffice:** If latency is critical (e.g., reactive safety behaviors), 5 DDIM steps at 5.6 ms may be the optimal operating point.
- **EMA is essential:** Evaluating with EMA weights (decay = 0.995, effective window = 200 gradient steps) consistently outperformed non-EMA evaluation by filtering out noisy weight updates without sacrificing final performance.

8. Conclusion

We presented a complete from-scratch implementation of Diffusion Policy for robot manipulation on PushT. Our evaluation of four methods confirms that distributional action policies dramatically outperform direct

regression in multi-modal expert settings. The **24× performance gap** between BC (4%) and Flow Matching (98%) is a direct empirical demonstration of the mode-averaging failure of BC.

Among diffusion methods, Flow Matching is the best choice: no noise schedule tuning, a strictly supervised velocity-matching objective, and the highest success rate (98%) at the lowest inference latency (8.9 ms).

Our most important engineering contribution is the DDIM numerical stability fix (clamping the cosine-schedule denominator at 10^{-3} and clipping predictions to $[-1, 1]$), which lifts DDIM from 0% to 92% on an otherwise well-trained model—a critical finding for practitioners implementing diffusion policies.

Finally, our inference-step ablation reveals that 5 denoising steps achieve 100% success—opening the door to real-time deployment at 5 ms per control step, well within the latency budget of physical robot systems.

Acknowledgements. We thank the Northeastern University Research Computing team for access to the Explorer HPC cluster (NVIDIA A100 and V100 SXM2 nodes) used to train all models. We also thank Chi et al. [1] for open-sourcing the Columbia PushT dataset and reference codebase.

References

- [1] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Morales, Homer Walke, Ken Goldberg, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [2] Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2022.
- [3] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [5] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Rectified flow from any distribution. In *International Conference on Learning Representations (ICLR)*, 2023.
- [6] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018.
- [7] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.
- [8] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.